

AG35-CET&AG35-EUT

QuecOpen(SDK)分区调整指导

LTE Standard 模块系列

版本：1.0.0

日期：2024-01-10

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

文档历史

修订记录

版本	日期	作者	变更表述
-	2024-01-10	Ethan WEN	文档创建
1.0.0	2024-01-10	Ethan WEN	临时版本

目录

文档历史	3
目录	4
表格索引	5
1 引言	6
2 分区调整说明.....	7
2.1. 分区介绍	7
2.2. 分区调整注意事项	8
3 分区调整方法.....	9
3.1. 调整 Flash Layout	10
3.1.1. 涉及文件	10
3.1.2. 调整方法	11
3.2. 配置 blf 文件	16
3.2.1. 涉及文件	16
3.2.2. 配置方法	16
3.3. 烧录固件	19
3.4. 添加挂载逻辑到新增分区	20
4 squashfs 和 UBI 固件制作和加载	23
4.1. oemapp.squashfs 制作和加载.....	23
4.1.1. 制作 squashfs 文件系统镜像	23
4.1.2. 加载 squashfs 文件系统	24
4.2. backup_data.ubi 制作.....	24
5 附录 参考文档及术语缩写	26

表格索引

表 1: 可调整的 Linux 分区概览.....	7
表 2: 重点需要调整的文件列表	8
表 3: update.blf 配置文件描述.....	10
表 4: blf 文件列表	16
表 5: 参考文档	26
表 6: 术语缩写	26

Preliminary
Quectel Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen[®]方案。QuecOpen[®]是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen[®]的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen[®]方案，主要介绍移远通信 AG35-CET 和 AG35-EUT 模块的分区调整方法以及相关注意事项。

Preliminary
Quectel Confidential

2 分区调整说明

2.1. 分区介绍

QuecOpen®方案下，Linux 系统中用于存放和读写用户应用程序以及配置参数数据的相关分区如下表所示。

表 1: 可调整的 Linux 分区概览

用户分区	默认空间	可用空间	分区格式	挂载点	分区用途
oemapp	20 MB	约 20 MB	squashfs 文件系统	/oemapp	带有 A/B 分区互为备份，支持 OTA 升级，用于存放客户的应用程序和配置参数数据。
oemdata	90 MB	约 73 MB	ubifs 文件系统	/oemdata	不带有 A/B 分区互为备份，不支持 OTA 升级，用于存放客户数据。
rootfs_data	13.75 MB	约 1 MB	ubifs 文件系统	/data、/mnt、/log	系统可读写分区
				/overlay/etc	
			overlayfs 文件系统	/overlay/nvm /system/etc /NVM	

备注

1. oemapp-a 和 oemapp-b 分区用于存放用户程序及配置文件，挂载为只读。oemapp 分区用作 A/B 分区，有备份还原功能，某个分区损坏后会切换系统进行还原恢复；也有 OTA 升级功能。因此建议使用该分区存放重要应用程序及配置文件。
2. rootfs_data 分区以可读写形式挂载，分为三个 UBI 卷，其中 0 号卷（ubi0_0）分别挂载到/data、/mnt 和/log 目录，另两个卷分别挂载到/system/etc 和/NVM 目录。rootfs_data 裸分区大小约 13.75 MB，可用大小只有约 1 MB，因为移远通信在该分区存放配置数据。且由于分区挂载失败时会擦除该分区 /system/etc 和/NVM 卷，所以不要直接使用该读写分区，而是使用 oemdata 分区。若需调整 oemdata 分区或可读写的 backup_data 分区，请参考第 3 章。

2.2. 分区调整注意事项

1. 除调整 oemapp、oemdata 和 rootfs_data 分区的大小及新增分区(如可读写的 backup_data 分区)外, 请勿修改其他分区。
2. 修改 oemapp 或 rootfs_data 分区大小:
 - oemapp 分区挂载在 /oemapp 目录下, 建议用户将应用程序存放在该分区以便管理。同时, oemapp 分区分为 A 和 B 分区, 修改 oemapp-a 分区大小时, 需同步修改 oemapp-b 分区。
 - rootfs_data 分区位于 flash layout 的最后位置(BBM 分区为系统预留的分区, 不参与分区 flash layout 的划分), 去除其他分区的占用空间之后, flash 的剩余空间都将分给 rootfs_data, 因此修改 oemapp-a/oemapp-b 分区大小后, 无需修改 rootfs_data 分区大小, 只需调整 rootfs_data 分区偏移地址即可, 但需保证 rootfs_data 分区能够正常挂载。
3. 新增分区:

若新增一个可读写的 backup_data 分区, 建议添加在 persist 分区之前。

表 2: 重点需要调整的文件列表

文件	描述
ql-1806e-flash-layout.dtsi	分区表, 位于 ql-ol-kernel/arch/arm/boot/dts/
81_mount_ubifs_overlay	开机挂载脚本, 位于 ql-ol-rootfs/lib/preinit/
ubinize-oemapp.cfg	UBI 配置文件, 位于 tools/conf/
Makefile	SDK 编译文件, 位于 SDK 根目录
update.blf	SWDownloader 配置文件, 位于固件包下的 update/目录
quectel_AB_OTA.blf	用于制作 OTA 升级包的配置文件, 位于 tools/fota/
quectel_skylark_pm802_standard_AB.blf	用于制作 OTA 升级包的配置文件, 位于 tools/fota/, 内容与 update.blf 相同

3 分区调整方法

配置文件 *update.blf* 中定义了镜像列表，和 SWDownloader 烧录界面中的镜像列表相对应。每个镜像包含了相应的镜像名称、ID 等属性。相关数据段如下所示：

```
[Image_List]
1_Image_Enable = 1
1_Image_Tim_Included = 1
1_Image_Image_ID = 0x54494D48
1_Image_Next_Image_ID = 0x54494D48
1_Image_Path = tim_fact_qspinand.bin
1_Image_Flash_Entry_Address = 0x00000000
1_Image_Load_Address = 0xD1000000
1_Image_Type = RAW
1_Image_ID_Name = TIMH
1_Image_Erase_Size =
1_Image_Partition_Number = 0
1_Image_Hash_Algorithm_ID = SHA-256
1_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
.....
20_Image_Enable = 1
20_Image_Tim_Included = 3
20_Image_Image_ID = 0x4F454D44
20_Image_Next_Image_ID = 0xFFFFFFFF
20_Image_Path = oemapp.squashfs
20_Image_Flash_Entry_Address = 0x06D20000
20_Image_Load_Address = 0xFFFFFFFF
20_Image_Type = RAW
20_Image_ID_Name = OEMD
20_Image_Partition_Number = 0
20_Image_Hash_Algorithm_ID = SHA-256
20_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
```

表 3: update.blf 配置文件描述

关键词	描述
Enable	是否下载此镜像。 1 下载 0 不下载
TIM_Included	镜像是否加载至 TIM 表。 0 不加载 1 加载至 tim 2 加载至 dtim-a 3 加载至 dtim-b 4 加载至 dtim CP 5 包含 TIM 和 OBM 的备份信息至 TIM 表
Image_ID	TIM 表中使用 ASCII 码识别的镜像。必须对应下述 ID_Name。
Next_Image_ID	下一个镜像的 Image_ID。最后一个镜像设置为 0xFFFFFFFF。
Path	镜像文件相对于 <i>update.blf</i> 的相对路径
Flash_Entry_Address	镜像烧录到 Flash 中的物理地址
Load_Address	镜像烧录到 DDR Memory 中对应的物理地址，默认只有 TIM、OBM、u-boot 和 tos 需要配置，其他镜像都设置为 0xFFFFFFFF。
Type	镜像格式，默认为 RAW。
ID_Name	镜像 ID 名称。必须是四个大写字母，必须对应上述 Image_ID。
Partition_Number	在 eMMC 闪存上的镜像分区位置，默认为 0。
Hash_Algorithm_ID	哈希算法，默认为 SHA-256。
Image_Size_To_Hash_in_bytes	需哈希的镜像大小，默认为 0xFFFFFFFF。实际镜像大小由下载工具计算。

3.1. 调整 Flash Layout

3.1.1. 涉及文件

以 AG35CETCAR01A02M2G_OCPU 版本为例，flash layout 文件为 *ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-kernel/arch/arm/boot/dts/ql-1806e-flash-layout.dtsi*

3.1.2. 调整方法

在相应文件中调整分区设备树 flash layout，如下所示：

```
&qspi {
    spinand: spinand@0 {
        partition@0 {
            label = "bootloader";
            reg = <0x0 0xA0000>;
            read-only;
        };

        partition@1 {
            label = "cp_reliabledata";
            reg = <0xA0000 0x20000>;
        };

        partition@2 {
            label = "ap_reliabledata";
            reg = <0xC0000 0x20000>;
        };

        partition@3 {
            label = "cp_reliabledata_backup";
            reg = <0xE0000 0x20000>;
        };

        partition@4 {
            label = "ap_reliabledata_backup";
            reg = <0x100000 0x20000>;
        };

        partition@5 {
            label = "mep-ota";
            reg = <0x120000 0x20000>;
        };

        partition@6 {
            label = "mep-ota_backup";
            reg = <0x140000 0x20000>;
        };

        partition@7 {
            label = "asr_flag";
            reg = <0x160000 0x100000>;
        };
    };
}
```

```

};

partition@8 {
    label = "cust_info";
    reg = <0x260000 0x80000>;
};

partition@9 {
    label = "misc";
    reg = <0x2E0000 0x80000>;
};

partition@10 {
    label = "dtim-a";
    reg = <0x360000 0x40000>;
};

partition@11 {
    label = "cpimage-a";
    reg = <0x3A0000 0xCE0000>;
};

partition@12 {
    label = "tos-a";
    reg = <0x1080000 0x100000>;
};

partition@13 {
    label = "u-boot-a";
    reg = <0x1180000 0xC0000>;
};

partition@14 {
    label = "kernel-a";
    reg = <0x1240000 0x800000>;
};

partition@15 {
    label = "rootfs-a";
    reg = <0x1A40000 0x1400000>;
};

partition@16 {
    label = "oemapp-a";

```

```

        reg = <0x2E40000 0x1400000>;
    };
    partition@17 {
        label = "dtim-b";
        reg = <0x4240000 0x40000>;
    };

    partition@18 {
        label = "cpimage-b";
        reg = <0x4280000 0xCE0000>;
    };

    partition@19 {
        label = "tos-b";
        reg = <0x4F60000 0x100000>;
    };

    partition@20 {
        label = "u-boot-b";
        reg = <0x5060000 0xC0000>;
    };

    partition@21 {
        label = "kernel-b";
        reg = <0x5120000 0x800000>;
    };

    partition@22 {
        label = "rootfs-b";
        reg = <0x5920000 0x1400000>;
    };

    partition@23 {
        label = "oemapp-b";
        reg = <0x6D20000 0x1400000>;
    };

    partition@24 {
        label = "oemdata";
        reg = <0x8120000 0x5A00000>;
    };

```

//若需新增分区，推荐在此处添加，并相应调整 **oemdata** 分区

```

    partition@25 {
        label = "persist";
    };

```

```

        reg = <0xDB20000 0xA00000>;
    };

    partition@26 {
        label = "rootfs_data";
        reg = <0xE520000 0>;
    };
};

```

- 新增分区

以新增一个大小为 16 MB (0x1000000) 的可读写分区 backup_data 为例，缩小 oemdata 空间，将缩小出来的空间分配给新增分区 backup_data:

```

partition@24 {
    label = "oemdata";
    reg = <0x8120000 0x4A00000>;
};

partition@25 {
    label = "backup_data";
    reg = <0xCB20000 0x1000000>;
};

partition@26 {
    label = "persist";
    reg = <0xDB20000 0xA00000>;
};

partition@27 {
    label = "rootfs_data";
    reg = <0xE520000 0>;
};

```

- oemapp 分区大小调整

以将 oemapp-a 分区的大小由 20 MB 增加到 30 MB 为例:

入口地址不变，分区大小由 0x1400000 增加到 0x1e00000，同时相应调整 oemapp-b 的大小；由于 oemapp-a 及 oemapp-b 分区大小的调整会导致后面其他分区的入口地址的变化，用户需进行相应调整。

调整前:

```
partition@16 {
    label = "oemapp-a";
    reg = <0x2E40000 0x1400000>;
};
partition@17 {
    label = "dtim-b";
    reg = <0x4240000 0x40000>;
};
partition@18 {
    label = "cpimage-b";
    reg = <0x4280000 0xCE0000>;
};
```

调整后:

```
partition@16 {
    label = "oemapp-a";
    reg = <0x2E40000 0x1e00000>;
};
partition@17 {
    label = "dtim-b";
    reg = <0x4c40000 0x40000>;
};
partition@18 {
    label = "cpimage-b";
    reg = <0x4c80000 0xCE0000>;
};
.....
```

//后面其他分区的入口地址均需用户进行相应调整

备注

调整完成后，需重新编译生成 **zImage** 镜像并替换到固件包中。编译方法参考 [文档 \[1\]](#)。

3.2. 配置 blf 文件

3.2.1. 涉及文件

表 4: blf 文件列表

文件名称	描述
<i>update.blf</i>	SWDownloader 配置文件，位于固件包的 <i>update/</i> 目录
<i>quectel_AB_OTA.blf</i>	OTA blf 配置文件，位于 SDK 的 <i>tool/fota/</i> 目录下
<i>quectel_skylark_pm802_standard_AB.blf</i>	标准版本 blf 配置文件，位于 SDK 的 <i>tool/fota/</i> 目录下，内容与 <i>update.blf</i> 完全相同

blf 文件是与下载工具 SWDownloader 配套使用的配置文件。其主要功能如下：

- 配置 DDR 参数
- 配置镜像列表
- 定义其他可配置参数

从使用角度，用户只需关注镜像列表（*image_list*）的配置。

3.2.2. 配置方法

以调整 oemapp 分区及新增 backup_data 分区为例，修改 *update.blf* 文件：

- 新增 backup_data 分区供挂载使用

若新增分区不需要作为镜像烧录，则无需在 *image_list* 中新增镜像配置；若新增分区需要作为镜像烧录，则需在 *image_list* 中新增镜像配置。

示例如下：

a. 修改镜像总数

修改前：

```
Number_of_Images = 20
```

修改后：

```
Number_of_Images = 21
```

b. 新增 image_list

```

20_Image_Enable = 1
20_Image_Tim_Included = 3
20_Image_Image_ID = 0x4F454D44
20_Image_Next_Image_ID = 0x44415441 //由 0xFFFFFFFF 改为 0x44415441
20_Image_Path = oemapp.squashfs
20_Image_Flash_Entry_Address = 0x06D20000
20_Image_Load_Address = 0xFFFFFFFF
20_Image_Type = RAW
20_Image_ID_Name = OEMD
20_Image_Partition_Number = 0
20_Image_Hash_Algorithm_ID = SHA-256
20_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
//////////新增下面这个 image list
21_Image_Enable = 1
21_Image_Tim_Included =
21_Image_Image_ID = 0x44415441 // Image_ID_Name 的 ASCII 码 (44-D, 41-A, 54-T)
21_Image_Next_Image_ID = 0xFFFFFFFF //如为最后镜像, 设为 0xFFFFFFFF
21_Image_Path = backup_data.ubi
21_Image_Flash_Entry_Address = //backup_data 分区的入口地址
21_Image_Load_Address = 0xFFFFFFFF
21_Image_Type = RAW
21_Image_ID_Name = DATA //自取一个和上述列表不冲突的名字
21_Image_Erase_Size = //backup_data 分区的大小, 可读写分区必填
21_Image_Partition_Number = 0
21_Image_Hash_Algorithm_ID = SHA-256
21_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF

```

● 调整 oemapp 分区

若调整 oemapp-a、oemapp-b 分区的大小, 需修改 *update.blf* 文件相关分区配置。

a. 调整 dtim 分区地址。因 dtim-b 分区的入口地址发生相应偏移, 需要修改:

```

3_Image_Map_Info
3_Image_ID = 0x54494D32
3_Image_Type = RECOVERYIMAGE //即 dtim-b
3_Flash_Address_Lo = 0x04240000 //dtim-b 分区入口地址
3_Flash_Address_Hi = 0x00000000
3_Partition = 0x00000000
3_Enable = 1
3_End_Image_Map_Info
4_Image_Map_Info
4_Image_ID = 0x54494D35

```

```
4_Image_Type = PPSETTINGIMAG_2
4_Flash_Address_Lo = 0x04260000 //基于 dtim-b 分区入口地址 + 0x20000
4_Flash_Address_Hi = 0x00000000
4_Partition = 0x00000000
4_Enable = 1
4 End Image Map Info
```

b. 此外，还需修改在 oemapp 分区后，所有镜像的分区入口地址：

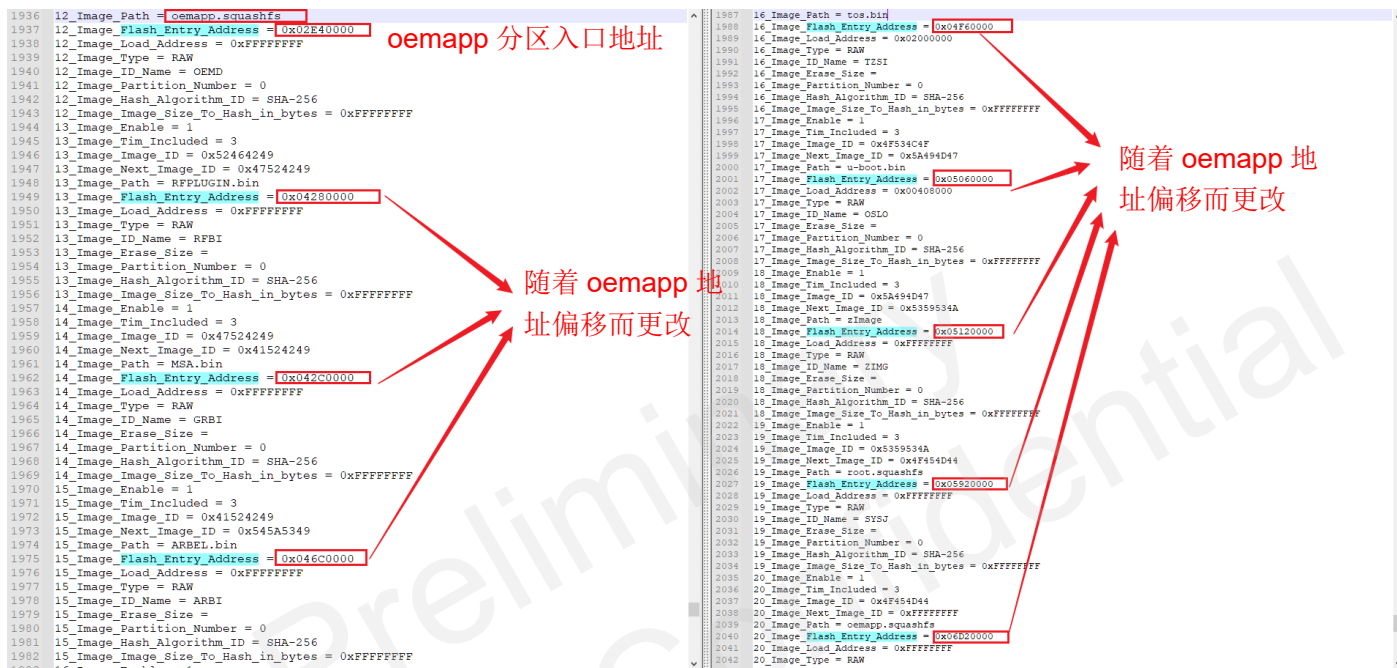


图 1: 修改 oemapp 分区后所有镜像的分区入口地址

c. 如果修改了 oemdata、rootfs_data 的分区地址或大小,则需要相应修改 EraseOnly_Option 部分。

如下所示，1 对应 `asr_flag` 分区，擦除是为了恢复其默认配置并确保烧录后进入的是 A 系统；2 对应 `oemdata` 分区；3 对应 `persist` 分区；4 对应 `rootfs_data` 分区。

```
[EraseOnly_Option]
Total_Eraseonly_Areas = 4
1_Eraseonly_Area_Size = 0x00100000
1_Eraseonly_Area_FlashStartAddress = 0x00160000
1_Eraseonly_Area_Partition = 0
1_Eraseonly_Area_Enable = 1
2_Eraseonly_Area_Size = 0x05A00000
2_Eraseonly_Area_FlashStartAddress = 0x08120000
2_Eraseonly_Area_Partition = 0
2_Eraseonly_Area_Enable = 1
3_Eraseonly_Area_Size = 0x00A00000
3_Eraseonly_Area_FlashStartAddress = 0x0DB20000
3_Eraseonly_Area_Partition = 0
3_Eraseonly_Area_Enable = 1
4_Eraseonly_Area_Size = 0x00dc0000
4_Eraseonly_Area_FlashStartAddress = 0x0E520000
4_Eraseonly_Area_Partition = 0
4_Eraseonly_Area_Enable = 1
```

备注

1. 如修改了分区表且需 OTA 升级，需修改 `quectel_skylark_pm802_standard_AB.blf` 和 `quectel_AB_OTA.blf` 制作 OTA 升级包，修改方式可参考 `update.blf` 文件的修改，也可使用 `tool/fota/part_resizing.sh` 脚本辅助修改。
2. 其中，`quectel_skylark_pm802_standard_AB.blf` 文件与 `update.blf` 文件内容一致，`quectel_AB_OTA.blf` 里的 `image_list` 的 `Image_Flash_Entry_Address` 应与 `update.blf` 的 `Image_Flash_Entry_Address` 一致。
3. `quectel_AB_OTA.blf` 里的 `image_list` 相比 `update.blf` 文件，缺少 `tim_falcon_qspinand.bin` 和 `TLoader_QSPINAND.bin` 两个镜像，为正常现象。
4. 如果新建的是可读写的分区，必须在 `EraseOnly_Option` 添加选项或在 `X_Image_Erase_Size` 指定镜像大小，从而在烧录时对该分区进行擦除，否则 UBI 会判定未擦除的部分为垃圾数据，或镜像已损坏。

3.3. 烧录固件

将 `zImage`（详见第 3.1.2 章）和 `blf` 文件（详见第 3.2.2 章）替换到固件包内并烧录至模块后，分区修改生效。执行 `cat /proc/mtd` 显示 MTD 分区信息，如下所示：

```
root@OpenWrt:~# cat /proc/mtd
dev:   size   erasesize  name
mtd0: 000a0000 00020000 "bootloader"
mtd1: 00020000 00020000 "cp_reliabledata"
mtd2: 00020000 00020000 "ap_reliabledata"
mtd3: 00020000 00020000 "cp_reliabledata_backup"
mtd4: 00020000 00020000 "ap_reliabledata_backup"
mtd5: 00020000 00020000 "mep-ota"
mtd6: 00020000 00020000 "mep-ota_backup"
mtd7: 00100000 00020000 "asr_flag"
mtd8: 00080000 00020000 "cust_info"
mtd9: 00080000 00020000 "misc"
mtd10: 00040000 00020000 "dtim-a"
mtd11: 00ce0000 00020000 "cpimage-a"
mtd12: 00100000 00020000 "tos-a"
mtd13: 000c0000 00020000 "u-boot-a"
mtd14: 00800000 00020000 "kernel-a"
mtd15: 01400000 00020000 "rootfs-a"
mtd16: 01e00000 00020000 "oemapp-a" // 分区大小由原来的 20 MB (0x01400000) 增加到现在的
                                     30 MB (0x01e00000)
mtd17: 00040000 00020000 "dtim-b"
mtd18: 00ce0000 00020000 "cpimage-b"
mtd19: 00100000 00020000 "tos-b"
```

```
mtd20: 000c0000 00020000 "u-boot-b"
mtd21: 00800000 00020000 "kernel-b"
mtd22: 01400000 00020000 "rootfs-b"
mtd23: 01e00000 00020000 "oemapp-b" // 分区大小由原来的 20 MB (0x01400000) 增加到现在的
                                     30 MB (0x01e00000)
mtd24: 03600000 00020000 "oemdata" // 分区大小由原来的 90 MB (0x05a00000) 减小到现在的
                                     54 MB (0x03600000)
                                     // 减少的 36 MB 一部分用于新增 backup_data 分区 (16 MB),
                                     另外一部分用于 oemapp-a 扩容 (10 MB) 及 oemapp-b 扩容
                                     (10 MB)
mtd25: 01000000 00020000 "backup_data" // 新增 backup_data 分区大小为 16 MB (0x01000000)
mtd25: 00a00000 00020000 "persist"
mtd26: 00dc0000 00020000 "rootfs_data"
mtd27: 00d20000 00020000 "BBM"
```

备注

由于调整分区对其后分区会产生影响并需相应修改，非必要不建议调整分区表。

3.4. 添加挂载逻辑到新增分区

需要挂载使用的分区，可以使用 `ql-ol-rootfs/lib/preinit/81_mount_ubifs_overlay` 脚本对其进行调整。以将新增 `backup_data` 分区挂载为 `ubifs` 文件系统为例：

1. 获取新增分区的 MTD 号：

```
ubifs_volume_support() {
    mtdpart_idx="$(find_mtd_index rootfs_data)"
    [ -z "$mtdpart_idx" ] && return 1
    + mtdpart_idx_backup_data="$(find_mtd_index backup_data)"
    + [ -z "$mtdpart_idx_backup_data" ] && return 1
    .....
}
```

2. 执行 `fakeroot mkdir backup_data` 命令，在 `ql-ol-rootfs/` 下创建挂载 `ubifs` 的可读写挂载目录 `backup_data`：

```
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu$ cd ql-ol-rootfs/
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-rootfs$ fakeroot mkdir backup_data
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-rootfs$ ls
backup_data  data  etc  log  NVM  oemdata  persist  rom  sbin  sys  tmp  var
bin          dev  lib  mnt  oemapp  overlay  proc  root  sdcard  system  usr
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-rootfs$ █
```

```
ubifs_backup_data_mount() {
    recover_ubifs=0    //recover_ubifs 为 0，挂载正常；recover_ubifs 为 1，检查挂载所需设备
                        节点是否异常，格式化并重新创建节点；
    [ ! -e /dev/ubi4 ] && ubiattach /dev/ubi_ctrl -m $mtdpart_idx_backup_data -d 4 ||
    recover_ubifs=1    //d 参数值代表第 4 个 UBI 设备；recover_ubifs 非 0 则 UBI 设备异常
    if [ $recover_ubifs -eq 0 ]
    then
        ubi4_nod_id=`cat /sys/class/ubi/ubi4/dev | tr -s ":" " "`
        [ ! -e /dev/ubi4 ] && mknod /dev/ubi4 c ${ubi4_nod_id}
        if [ ! -e /sys/class/ubi/ubi4_0/dev ]
        then
            # no volume
            recover_ubifs=1    //卷设备异常
        else
            # check for "backup_data" volume
            ubi4_0_nod_id=`cat /sys/class/ubi/ubi4_0/dev | tr -s ":" " "`
            [ ! -e /dev/ubi4_0 ] && mknod /dev/ubi4_0 c ${ubi4_0_nod_id}
            { ubinfo /dev/ubi4_0 | grep Name | grep -qs "backup_data" ; } || \
            recover_ubifs=1    //卷设备节点异常
        fi
    fi
    if [ $recover_ubifs -eq 1 ]    ///检查设备及节点异常后进行分区格式化重新创建修复
    then
        echo "ubifs data partition is damaged"
        echo "try to recover by formatting $mtdpart..."
        [ -e /dev/ubi4 ] && ubidetach -m $mtdpart_idx_backup_data
        ubiformat -y -q /dev/mtd$mtdpart_idx_backup_data    //格式化 MTD
        ubiattach -m $mtdpart_idx_backup_data /dev/ubi_ctrl    //创建 UBI 设备
        ubi4_nod_id=`cat /sys/class/ubi/ubi4/dev | tr -s ":" " "`
        [ ! -e /dev/ubi4 ] && mknod /dev/ubi4 c ${ubi4_nod_id}
        ubimkvol /dev/ubi4 -n 0 -N backup_data -t dynamic --maxavsize    //创建卷设备
    fi
    mount -t ubifs -o rw,noatime,bulk_read ubi4:backup_data /backup_data    //挂载
    return 0
}
```

3. 调用 `ubifs_backup_data_mount()`:

```
ubifs_syscfg_rootfs_pivot() {
    .....
    # for nvm/gki file override
    mkdir -p /NVM/oem_data
    + ubifs_backup_data_mount    //调用挂载函数
}
```

挂载完成后，新增 backup_data 分区的状态如下所示：

```

root@OpenWrt:~# df -Th && mount
Filesystem      Type      Size      Used Available Use% Mounted on
/dev/root       squashfs  16.4M     16.4M      0 100% /
tmpfs           tmpfs     109.1M    120.0K    109.0M   0% /tmp
ubi0:data       ubifs     1.1M      28.0K      1.0M   3% /data
ubi0:data       ubifs     1.1M      28.0K      1.0M   3% /mnt
ubi0:data       ubifs     1.1M      28.0K      1.0M   3% /log
ubi0:etc        ubifs     2.9M      52.0K      2.7M   2% /overlay/etc
ubi0:nvm        ubifs     2.0M      56.0K      1.8M   3% /overlay/nvm
overlayfs:/overlay/etc overlay    2.9M      52.0K      2.7M   2% /system/etc
overlayfs:/overlay/nvm overlay    2.0M      56.0K      1.8M   3% /NVM
/dev/mtdblock16 squashfs  128.0K    128.0K      0 100% /oemapp
ubi2:data       ubifs     63.6M     24.0K     60.3M   0% /oemdata
ubi3:persist    ubifs     6.3M      20.0K      5.9M   0% /persist
ubi4:backup_data ubifs     11.6M     20.0K     11.0M   0% /backup_data
tmpfs           tmpfs     512.0K      0     512.0K   0% /dev
/dev/root on / type squashfs (ro,seclabel,relatime)
sysfs on /sys type sysfs (rw,seclabel,relatime)
selinuxfs on /sys/fs/selinux type selinuxfs (rw,nosuid,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,noatime)
tmpfs on /tmp type tmpfs (rw,seclabel,nosuid,nodev,noatime)
ubi0:data on /data type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=0)
ubi0:data on /mnt type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=0)
ubi0:data on /log type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=0)
ubi0:etc on /overlay/etc type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=1)
ubi0:nvm on /overlay/nvm type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=2)
overlayfs:/overlay/etc on /system/etc type overlay (rw,seclabel,noatime,lowerdir=/system/etc,upperdir=/ov
overlayfs:/overlay/nvm on /NVM type overlay (rw,seclabel,noatime,lowerdir=/NVM,upperdir=/overlay/nvm/root
/dev/mtdblock16 on /oemapp type squashfs (ro,seclabel,relatime)
ubi2:data on /oemdata type ubifs (rw,seclabel,noatime,assert=read-only,ubi=2,vol=0)
ubi3:persist on /persist type ubifs (rw,seclabel,noatime,assert=read-only,ubi=3,vol=0)
ubi4:backup_data on /backup_data type ubifs (rw,seclabel,noatime,bulk_read,assert=read-only,ubi=4,vol=0)
tmpfs on /dev type tmpfs (rw,seclabel,nosuid,relatime,size=512k,mode=755)
devpts on /dev/pts type devpts (rw,seclabel,nosuid,noexec,relatime,mode=600,ptmxmode=000)
debugfs on /sys/kernel/debug type debugfs (rw,seclabel,noatime)
none on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,noatime,mode=700)

```

4 squashfs 和 UBI 固件制作和加载

本章以 AG35CETCAR01A02M2G_OCPU 版本为例介绍 squashfs 和 UBI 固件的制作和加载方法。

4.1. oemapp.squashfs 制作和加载

用户使用 oemapp 分区将应用程序及配置文件打包进 *oemapp.squashfs* 固件并烧录至模块。squashfs 文件系统的镜像制作和加载方法如下所示：

4.1.1. 制作 squashfs 文件系统镜像

解压 SDK 后进入 *ql-ol-extsdk-ag35cetcar01a02m2g_ocpu* 目录下，执行 **make oemapp** 创建 *oemapp/* 目录。该目录下有 *config/* 目录用于存放 GKI 相关配置，用户无需关注。将用户数据存入 *oemapp/* 目录下，然后执行 **make oemapp** 在 *target/* 目录下生成 *oemapp.squashfs*。将生成的 *oemapp.squashfs* 新固件替换到固件包内进行烧录，或通过 FOTA 方式进行烧录。

```
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu$ ls
firmware  oemapp      ql-ol-kernel  ql-ol-rootfs.tar.gz  ql-sysroots  sdk.mk  tools
Makefile  ql-ol-bootloader  ql-ol-rootfs  ql-selinux  sample      target
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu$ make oemapp
/home/ethan/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/tools/bin/fakeroot /home/ethan/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/tools/bin/mksquashfs4 /home/ethan/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/oemapp /home/ethan/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/target/oemapp.squashfs -nopad -noappend -root-owned -comp xz -Xpreset 8 -Xbcj arm -b 128k -processors 1;
Parallel mksquashfs: Using 1 processor
Creating 4.0 filesystem on /home/ethan/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/target/oemapp.squashfs, block size 131072.
===== 1/1 100%

Exportable Squashfs 4.0 filesystem, xz compressed, data block size 131072
  compressed data, compressed metadata, compressed fragments,
  compressed xattrs, compressed ids
  duplicates are removed
Filesystem size 0.53 Kbytes (0.00 Mbytes)
  99.09% of uncompressed filesystem size (0.54 Kbytes)
Inode table size 66 bytes (0.06 Kbytes)
  100.00% of uncompressed inode table size (66 bytes)
Directory table size 33 bytes (0.03 Kbytes)
  100.00% of uncompressed directory table size (33 bytes)
Number of duplicate files found 0
Number of inodes 2
Number of files 1
Number of fragments 1
Number of symbolic links 0
Number of device nodes 0
Number of fifo nodes 0
Number of socket nodes 0
Number of directories 1
Number of ids (unique uids + gids) 1
Number of uids 1
  root (0)
Number of gids 1
  root (0)
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu$ cd target && ls
oemapp.squashfs  root.squashfs
ethan@FS-D-000658:~/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/target$
```

4.1.2. 加载 squashfs 文件系统

squashfs 文件系统的加载，第一步是创建设备节点，第二步是 squashfs 文件系统的挂载。在系统重启时执行 `ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-rootfs/lib/preinit/81_mount_ubifs_overlay` 脚本即可实现此过程。

4.2. backup_data.ubi 制作

1. 修改 Makefile，新增 backup_data 分区路径及相关命令。

`mkfs.ubifs` 命令中，`-c` 参数表示最大的逻辑擦除块数，即卷大小除以物理擦除块大小。例如，卷大小为 10 MiB，则 `-c` 的值为 $10 \times 1024 / 128 = 80$ 。

```
export QL_SAMPLE_DIR=$(QL_SDK_DIR)/sample
export QL_TARGET_DIR=$(QL_SDK_DIR)/target
export OL_OEMAPP_DIR=$(OL_SDK_DIR)/oemapp
export QL_BACKUP_DIR=$(QL_SDK_DIR)/backup_data
```

```
#####
#                               #
#                               #
#####
.PHONY:oemapp
oemapp: pre_build
$(QL_TOOLS_BIN_DIR)/fakeroot $(QL_TOOLS_BIN_DIR)/mksquashfs $(QL_OEMAPP_DIR) $(QL_TARGET_DIR)/oemapp.squashfs -nopad -noappend -root-owned -comp

.PHONY:backup_data
backup_data:
mkdir -p $(QL_TARGET_DIR); \
mkdir -p $(QL_BACKUP_DIR); \
mkdir -p $(QL_BACKUP_DIR)/config; \
fakeroot $(QL_TOOLS_BIN_DIR)/mkfs.ubifs -r $(QL_BACKUP_DIR) -o $(QL_TARGET_DIR)/backup_data.ubifs -m 2048 -e 126976 -c 80 -F; \
cp $(QL_TOOLS_CONF_DIR)/ubinize-backup_data.cfg $(QL_TARGET_DIR)
cd $(QL_TARGET_DIR); \
fakeroot $(QL_TOOLS_BIN_DIR)/ubinize -o $(QL_TARGET_DIR)/backup_data.ubi -m 2048 -p 128KiB -s 2048 $(QL_TARGET_DIR)/ubinize-backup_data.cfg
```

2. 参考 `ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/tools/conf/ubinize-oem.cfg` 在同级目录下新建 `ubinize-backup_data.cfg`:

```
1 [backup_data]
2 # Volume mode (other option is static)
3 mode=ubi
4
5 # Source image
6 image=backup_data.ubifs
7
8 # Volume ID in UBI image
9 vol_id=0
10
11 # Allow for dynamic resize
12 vol_type=dynamic
13
14 # Volume name
15 vol_name=backup_data
16
17 # volume size
18 #vol_flags=autoresize
19 vol_size=9424KiB
```

其中 vol_size 计算方法如下:

逻辑擦除块总数 = backup_data 分区大小/物理擦除块大小; vol_size = (逻辑擦除块总数 - 4) × 逻辑擦除块大小。其中, 减去的 4 个块 2 个用于内部卷, 1 个用于磨损平衡, 1 个用于擦洗。

例如, 若 backup_data 分区大小为 10 MiB, 物理擦除块大小为 128 KiB, 逻辑擦除块大小为 124 KiB, 则 vol_size = [(10 × 1024)/128 - 4] × 124 = 9424 KiB。

3. 在 `ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/` 根目录下执行 **make backup_data** 生成 `backup_data.ubi`:

```
ethan@FS-D-000658:~/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu$ make backup_data
mkdir -p /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/target; \
mkdir -p /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/backup_data; \
mkdir -p /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/backup_data/config; \
fakeroot /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/tools/bin/mkfs.ubifs -r /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/backup_data -o /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/target/backup_data.ubifs -m 2048 -e 126976 -c 80 -F; \
cp /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/tools/conf/ubinize-backup_data.cfg /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/target
cd /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/target; \
fakeroot /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/tools/bin/ubinize -o /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/target/backup_data.ubi -m 2048 -p 128KiB -s 2048 /home/ethan/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/target/ubinize-backup_data.cfg
```

生成的 UBI 固件位于 `ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/target/` 目录下:

```
ethan@FS-D-000658:~/1806/quectel_build/sdk/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu$ ls target/
backup_data.ubi backup_data.ubifs ubinize-backup_data.cfg
```

5 附录 参考文档及术语缩写

表 5: 参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导

表 6: 术语缩写

缩写	英文全称	中文全称
ASCII	American Standard Code for Information Interchange	美国信息交换标准码
CP	Communication Processer	通信处理器
DDR	Double Data Rate	双倍数据速率
eMMC	Embedded Multimedia Card	嵌入式多媒体卡
FOTA	Firmware Over-The-Air	固件空中升级
GKI	General Kernel Image	通用内核镜像
ID	Identifier	标识符
IoV	Internet of Vehicles	车联网
MTD	Memory Technology Device	存储器技术设备
OBM	Original Brand Manufacturer	原始品牌制造商
SDK	Software Development Kit	软件开发工具包
TIM	Trusted Image Module	可信镜像模块
UBI	Unsorted Block Image	无排序区块镜像